DATABASE DESIGN · TUTORING

Database Design — Hands-On Practice Lesson

database-design-practice-lesson.txt

```
================================================================================
 DATABASE DESIGN - HANDS-ON PRACTICE LESSON
 Turning the rules into skill, in a free in-browser database (no install)
================================================================================

 Do this lesson AFTER the primer and the companion lesson. This is the first
 time you will actually touch a computer. You will build small tables, put rows
 in them, ask them questions, make the classic beginner mistakes ON PURPOSE,
 feel why they hurt, and then fix them. Reading taught you the ideas; this makes
 them yours.

 Nothing here is timed and nothing here is graded. Every table you make lives in
 a throwaway database inside a browser tab. You can break it, reload it, and
 start again as many times as you like. That freedom is the whole point: a
 beginner learns a rule best by watching the database ENFORCE it, right in front
 of them - and the only way to watch that is to break the rule on purpose, in a
 place where breaking it costs nothing.

 HOW TO USE THIS LESSON (please read this; it changes how much you get out of it)
   1. Keep three things open at once if you can - this lesson, the sandbox tab,
      and a sheet of paper. Design on paper first, type second, read the result
      third.
   2. Run SQL ONE STATEMENT AT A TIME. Type a block, run it, READ what came
      back, and only then move to the next block. The whole skill is in reading
      the result, not in typing fast.
   3. Every TRY IT below is followed by a drawn picture of what you should see -
      either a RESULT grid or an ERROR box. Compare your real screen to the
      picture. If they differ, THAT difference is the lesson: copy both what you
      typed and what you saw, and bring them to me.
   4. You do not need to understand SELECT or JOIN deeply yet. We type only
      enough query to make a DESIGN idea visible. The head (design) leads; the
      hands (SQL) follow.

 WHY A SANDBOX, AND WHY NOT INSTALL ANYTHING YET
 We agreed installation is the LAST phase. So this lesson uses a database that
 runs INSIDE A WEB BROWSER TAB - nothing to install, nothing that can touch your
 real machine or the student-panel project. When the tab closes, the practice
 database is gone. That is exactly what we want for practice: a safe sandbox you
 cannot break anything important in.

 Signposts are the same as always:
   >> CONTEMPLATE   stop and think first.
   TRY IT          type this into the sandbox and watch what happens.
   KEY IDEA        the one sentence to keep.
   >> PITFALL       a trap, named so you can step around it.
   LOOK FOR        what you should SEE after a TRY IT (so you know it worked).
   MAPS TO         where this exercise lives in the primer / samples PDF, and
                   how student-panel does the same thing (sometimes differently).
 Two more words this fuller lesson leans on (still plain English):
   BUILD / BREAK / FIX / TEST   the four beats of every exercise. You will see
                   these words called out so you always know which beat you are
                   on. We BUILD a thing, BREAK it the way a beginner would,
                   then FIX the design and TEST that the fix holds.
   RESULT / ERROR  the two kinds of drawn picture below. RESULT is the grid a
                   SELECT returns. ERROR is the exact refusal the database
                   raises when you break a rule. An ERROR here is a SUCCESS: it
                   means the database protected its own truth, which is the
```

whole reason we add rules in the first place.

There is no clock. If an exercise does not behave, copy what you typed and what you saw, and bring it to me. A broken query is a GREAT thing to learn from.

PART 0 - SETTING UP YOUR SANDBOX (read all of this before you start)

You need a web page that lets you type SQL and see the result. You have two good options. Read both; pick one.

OPTION A (preferred - it matches student-panel's real engine):
student-panel stores its data with something called PGlite, which is the PostgreSQL database compiled to run in a browser. Practicing on the same engine means the SQL you learn here is the SQL student-panel actually speaks. To find it: open a search engine and search for PGlite playground (the project's home is pglite.dev). Look for an in-browser REPL / playground where you can type SQL and run it. If you land on a page with a box that says something like "postgres=>" and lets you type, you are in the right place.

OPTION B (fallback - simplest, always available):
Any free online SQL playground. Search for SQLite online or SQL fiddle. These give you an instant SQL box with no sign-up. SQLite speaks a SLIGHTLY different dialect than Postgres (I point out every difference as we go), but every design idea in this lesson works identically in both.

>> CONTEMPLATE: notice we are choosing a tool the way the primer says to choose anything - by purpose. The purpose here is "practice the design safely, without installing, close to the real engine." Option A serves it best; Option B serves it good-enough. Either is fine for learning.

THREE DIALECT NOTES you will need (keep them nearby):

- AUTO-GENERATED IDS. In Postgres (Option A) an auto-numbered id column is written `id INTEGER GENERATED ALWAYS AS IDENTITY` (older style: SERIAL). In SQLite (Option B) it is `id INTEGER PRIMARY KEY` (SQLite fills it in for you) or `id INTEGER PRIMARY KEY AUTOINCREMENT`. To avoid this fork entirely while learning, THIS LESSON TYPES THE IDS BY HAND (1, 2, 3). That keeps every statement identical in both tools and keeps your attention on the DESIGN, not on id machinery.
- DATES. Both tools accept dates written as text like '2026-09-01'. Because that layout sorts the same as the calendar (year, then month, then day), plain text comparison such as `joined_on >= '2026-09-03'` works correctly. Good enough for practice.
- FOREIGN KEYS MUST BE SWITCHED ON IN SQLITE. This is the single most important dialect note in the whole lesson, so it comes first and loudest. In Postgres/PGlite (Option A) foreign-key rules are always enforced. In many SQLite playgrounds (Option B) they are OFF by default, which means the database will wrongly ALLOW the mistakes we are trying to feel. Before you start, run this once at the top of your SQLite tab and leave it:

```
PRAGMA foreign_keys = ON;
```

  If an exercise says "the database REFUSES this" and yours quietly accepts it, this pragma is almost certainly the reason. Option A users can ignore this line.

THE THREE DATA TYPES WE USE (that is all you need for this whole lesson):

Type	Holds	Example value (typed as)
INTEGER	whole numbers	1, 2, 999
TEXT	words / codes / dates	'Sara Ahmadi', 'C-001', '2026-09-01'

```
  | REAL    | numbers with a decimal | 3.5, 72.0                          |
  +---------+------------------------+------------------------------------+
```

Note that text values go in 'single quotes' and numbers do not. A date is just text in disguise here, so it is quoted too.
HOW TO READ THE DRAWN RESULTS IN THIS LESSON. After almost every TRY IT you will see one of two pictures. Learn to read them now, once:

A RESULT grid looks like this (it is what a SELECT hands back):

```
      +-----------+--------------+
      | member_id | full_name    |     <- the column headers
      +-----------+--------------+
      | 1         | Sara Ahmadi  |     <- one row that matched
      | 2         | Omid Karimi  |     <- another row that matched
      +-----------+--------------+
      (2 rows)                         <- how many rows came back
```

An ERROR box looks like this (it is the database REFUSING something):

```
      ERROR (Option A, Postgres / PGlite):
        ERROR:  some clear sentence about what rule you broke
      ERROR (Option B, SQLite):
        Error: some shorter sentence about the same rule
```

Your tool will show ONE of those two error wordings depending on which option you picked. They mean the same thing. When you see an ERROR box in this lesson, the point is NOT that something went wrong - it is that the database did its job.

THE SIX RULES ("CONSTRAINTS") YOU WILL MEET. Every one of these is a promise you ask the database to keep. We build them up across the lesson; here they are on one card so you can see where we are heading:

```
  +---------------+-----------------------------------------+-------------+
  | constraint    | the promise it makes                    | first met   |
  +---------------+-----------------------------------------+-------------+
  | PRIMARY KEY   | every row has one unique, never-null id | E1          |
  | NOT NULL      | this column may never be left empty     | E1          |
  | UNIQUE        | no two rows may share this value        | E1          |
  | FOREIGN KEY   | this pointer must aim at a row that exists | E5 / E6     |
  | ON DELETE ..  | what happens to children when a parent goes| E6          |
  | composite key | a COMBINATION of columns must be unique | E7          |
  +---------------+-----------------------------------------+-------------+
```

A CLEANUP RECIPE. The sandbox remembers your tables until you reload the tab. Reloading wipes everything back to empty - the fastest reset. If you would rather clear just the practice tables by hand, you can run:

```
    DROP TABLE IF EXISTS member;
    DROP TABLE IF EXISTS member_bad_phones;
    DROP TABLE IF EXISTS member_bad_list;
    DROP TABLE IF EXISTS student_bad_subjects;
    DROP TABLE IF EXISTS loan_wide;
```

(We add more DROPs as we create more tables; a full list sits in Part 6.)

>> PITFALL: do not reuse table names between exercises in the same tab. If a later exercise says CREATE TABLE member and you already made one, either start a fresh tab or run DROP TABLE member; first. The sandbox remembers until you reload.

A SAFETY NOTE: this is a scratch database in a browser tab. Type freely, break

things on purpose, reload to wipe it. Nothing here can reach your computer, your files, or the student-panel project.

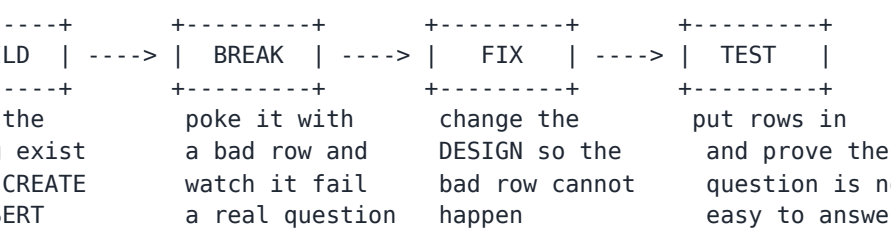
PART 1 - THE BIG PICTURE: WHAT WE WILL DO, AND WHY, BEFORE ANY TYPING

The whole lesson has one shape, biggest-picture first:

> BUILD a tiny good table -> BREAK it the way beginners do -> FEEL the pain -> FIX it with the design rules -> PROVE the fix with real rows.

That order is deliberate. If I only showed you the right answer, you would believe me but not UNDERSTAND. By making the mistake first and feeling exactly how it bites, the fix stops being a rule you memorised and becomes a reflex. This mirrors how the primer and the samples PDF teach normal forms: show the anomaly, then remove it.

THE FOUR BEATS, drawn as a loop you will walk eight times (once per exercise):



Not every exercise lands on all four beats equally - some are mostly BREAK (Part 3, where we commit the sins) and some are mostly FIX (Part 4, where we repair them) - but the loop is always the frame. Watch the beat labels.

Here is the arc of the five working parts, so you always know where you are:

- PART 2 WARM-UP - one table, three rows, your first question, and your first three refusals (NOT NULL, UNIQUE, PRIMARY KEY).
- PART 3 FEEL THE PAIN - commit the three classic sins (repeating columns, a list in one cell, one giant wide table) and watch them fail a real question.
- PART 4 FIX IT - split tables, add a foreign key, see the three ON DELETE rules act, build a bridge for a many-to-many, and a self-reference tree - plus three rarer patterns (E9-E11). The core of the lesson.
- PART 5 TEST WITH ROWS - the discipline both the primer and the PDF end on: put real rows in and answer your purpose questions.
- PART 6 MAP BACK - line every exercise up against the primer, the samples PDF, and student-panel (and where you differ).

ONE SMALL WORLD, USED ALL THE WAY THROUGH. So the exercises join up instead of feeling like scattered toys, we keep ONE little library and school running across the whole lesson. The same people and things reappear:

- MEMBERS of a library: Sara Ahmadi, Omid Karimi, Lena Hassan.
- Their PHONES (some have several - that is the point of E2/E3/E5).
- BOOKS: 'The Blue Door', 'Quiet Rivers', 'Open Sky'.
- LOANS: who borrowed which book, and when.
- and, for the many-to-many and the tree, STUDENTS taking COURSES and a tree of GROUPS/CATEGORIES like 'Grade 10 - Section A'.

Because the rows are the same from E1 onward, by Part 5 you can ask real questions ("how many loans does each member have?") and the answer joins rows you created pages earlier. Keep your tab open across exercises where you can.

HOW THE EXERCISES BUILD ON EACH OTHER (so a stumble early is worth fixing before moving on):

step	what it leaves you with	which later step reuses it
E1	a good 'member' table	the shape every other table copies
E2	the phone1/2/3 mistake	fixed in E5
E3	the list-in-a-cell sin	the phone list fixed in E5; the subjects list fixed in E7
E4	the wide-table anomaly	fixed in E6
E5	member2 + phone (FK)	member2 is reused by E6 and Part 5
E6	book + loan + ON DELETE	loans counted in Part 5 Q1 and Q5
E7	student/course/enrolment	grades read back in Part 5 Q2 and Q6
E8	the category tree	sub-groups listed in Part 5 Q3

KEY IDEA: you are not here to learn SQL. You are here to make the DESIGN IDEAS physical. SQL is just the hands; design is the head. We type only enough SQL to make the ideas move.

MAPS TO: this head-before-hands order is the primer's three LEVELS made real - your paper sketch is the CONCEPTUAL level (ideas in plain words), each CREATE TABLE is the LOGICAL level (tables, columns, keys), and the engine you picked in Part 0 (PGlite or SQLite) is the PHYSICAL level (real storage on disk). Beneath all of it sits the primer's biggest idea: a database is a chosen MODEL of a slice of reality - you keep the facts your questions need and drop the rest. The primer's naming mnemonic travels with our one small world too: the NOUNS become tables (member, book, loan), their ADJECTIVES become columns (full_name, borrowed_on), and the VERBS become the links between tables (a member BORROWS a book - that verb is the loan row). See primer Part 1 (the big picture, the three levels) and Part 2 (the three primitives).

>> CONTEMPLATE before you start: in one sentence, what question would make a badly-designed table obviously bad? (Hint from the primer: "a design is good if it can ANSWER the questions you need.") Hold your sentence; by Part 3 you will have made a table that cannot answer its own question.

PART 2 - WARM-UP: YOUR FIRST TABLE, ROWS, AND QUESTION

WHAT we will do: make one table, put three rows in it, ask it one question, and then try to break it three ways so you meet your first refusals.
WHY: to prove the sandbox works and to SEE the paper-to-grid-to-SQL chain from the companion lesson happen in front of you - and to feel, gently, that a table is not just a grid but a grid WITH RULES.

>> CONTEMPLATE: on paper, write the columns a "member" needs for a tiny library: an id, a full name, the date they joined, and one more - a membership card number. Why the card number? Because it is the one human-facing value that must never be shared by two members. That is our excuse to practise the UNIQUE rule, and it is exactly the role student-panel's unique student_code plays.

E1. CREATE, INSERT, SELECT.

-- BUILD ----------

TRY IT - type these one block at a time (run, read the result, then the next):

```
CREATE TABLE member (
  member_id  INTEGER PRIMARY KEY,
  full_name  TEXT NOT NULL,
  card_no    TEXT NOT NULL UNIQUE,
  joined_on  TEXT
);
```

```
    INSERT INTO member (member_id, full_name, card_no, joined_on) VALUES
      (1, 'Sara Ahmadi', 'C-001', '2026-09-01'),
      (2, 'Omid Karimi', 'C-002', '2026-09-03'),
      (3, 'Lena Hassan', 'C-003', '2026-09-05');

    SELECT * FROM member;

LOOK FOR: the SELECT shows a grid of three rows and four columns -

    +-----------+-------------+---------+------------+
    | member_id | full_name   | card_no | joined_on  |
    +-----------+-------------+---------+------------+
    | 1         | Sara Ahmadi | C-001   | 2026-09-01 |
    | 2         | Omid Karimi | C-002   | 2026-09-03 |
    | 3         | Lena Hassan | C-003   | 2026-09-05 |
    +-----------+-------------+---------+------------+
    (3 rows)

You just turned a paper design into a real, queryable thing.
READ YOUR OWN CREATE TABLE, line by line - this is the whole grammar of design
in six lines:
  - member_id INTEGER PRIMARY KEY  -> the id column; PRIMARY KEY means "this is
    the one unique handle for a row, and it can never be empty."
  - full_name TEXT NOT NULL        -> a name is text, and NOT NULL means a
    member with no name is forbidden.
  - card_no TEXT NOT NULL UNIQUE   -> text, required, and UNIQUE means no two
    members may carry the same card number.
  - joined_on TEXT                 -> a date-as-text; no NOT NULL, so it is
    allowed to be empty (maybe we do not always know the join date).

TRY IT - now ask a real question:

    SELECT full_name FROM member WHERE joined_on >= '2026-09-03';

LOOK FOR: two names -

    +-------------+
    | full_name   |
    +-------------+
    | Omid Karimi |
    | Lena Hassan |
    +-------------+
    (2 rows)

You asked "who joined on or after the 3rd?" and the table answered. THAT is
what "a design can answer the question" means, made concrete. (Sara joined on
the 1st, so she is correctly left out.)

>> PITFALL: forgetting the semicolon at the end of a statement. Most tools need
it to know you are done. If nothing happens, check for a missing  ;  .
-- BREAK (three refusals, on purpose) ------------------------------------

A table is a grid WITH RULES. Let us poke each rule and watch it hold. None of
these three inserts will add a row; each is meant to be REFUSED.

TRY IT (break 1 - NOT NULL): try to add a member with no name.

    INSERT INTO member (member_id, full_name, card_no, joined_on)
    VALUES (4, NULL, 'C-004', '2026-09-07');

LOOK FOR: a refusal, because full_name is NOT NULL -
```

```
    ERROR (Option A, Postgres / PGlite):
      ERROR:  null value in column "full_name" of relation "member"
      violates not-null constraint
    ERROR (Option B, SQLite):
      Error: NOT NULL constraint failed: member.full_name

TRY IT (break 2 - UNIQUE): try to reuse Sara's card number 'C-001'.

    INSERT INTO member (member_id, full_name, card_no, joined_on)
    VALUES (4, 'Nilou Rad', 'C-001', '2026-09-07');

LOOK FOR: a refusal, because card_no is UNIQUE and 'C-001' is already taken -

    ERROR (Option A, Postgres / PGlite):
      ERROR:  duplicate key value violates unique constraint
      "member_card_no_key"
      DETAIL:  Key (card_no)=(C-001) already exists.
    ERROR (Option B, SQLite):
      Error: UNIQUE constraint failed: member.card_no

TRY IT (break 3 - PRIMARY KEY): try to reuse member_id 1 (Sara's).

    INSERT INTO member (member_id, full_name, card_no, joined_on)
    VALUES (1, 'Nilou Rad', 'C-004', '2026-09-07');

LOOK FOR: a refusal, because member_id is the PRIMARY KEY and 1 already exists -

    ERROR (Option A, Postgres / PGlite):
      ERROR:  duplicate key value violates unique constraint "member_pkey"
      DETAIL:  Key (member_id)=(1) already exists.
    ERROR (Option B, SQLite):
      Error: UNIQUE constraint failed: member.member_id
-- TEST (the guards held; the data is unharmed) ---------------------------

TRY IT: look at the table again after those three refusals.

    SELECT * FROM member;

LOOK FOR: STILL exactly three rows - none of the bad inserts got in.

    +-----------+-------------+---------+------------+
    | member_id | full_name   | card_no | joined_on  |
    +-----------+-------------+---------+------------+
    | 1         | Sara Ahmadi | C-001   | 2026-09-01 |
    | 2         | Omid Karimi | C-002   | 2026-09-03 |
    | 3         | Lena Hassan | C-003   | 2026-09-05 |
    +-----------+-------------+---------+------------+
    (3 rows)

A rejected statement leaves the table exactly as it was - the database never
half-accepts a bad row. That all-or-nothing behaviour is a quiet promise you
will lean on constantly.

-- A NOTE ON ATTRIBUTE KINDS (meet the fourth one) ------------------------

Primer Part 2 names four kinds of attribute, and this lesson drills all four.
Three you meet head-on: ATOMIC (one value per cell - the whole point of E3),
MULTI-VALUED (many per owner - the phones of E2/E3/E5), and DERIVED (computed,
not stored - the counts in E5 and Part 5). The fourth, COMPOSITE (one name
hiding several values - a full name = given + family, an address = street + city
+ postcode), deserves one quick TRY IT, because full_name above quietly chose
NOT to split.
```

TRY IT - store the name as two columns, which lets you sort by family name (a single lumped full_name cannot do this cleanly):

```
    CREATE TABLE member_split (
      member_id   INTEGER PRIMARY KEY,
      given_name  TEXT NOT NULL,
      family_name TEXT NOT NULL
    );

    INSERT INTO member_split VALUES
      (1, 'Sara', 'Ahmadi'),
      (2, 'Omid', 'Karimi'),
      (3, 'Lena', 'Hassan');

    SELECT given_name, family_name FROM member_split ORDER BY family_name;
```

LOOK FOR: rows in family-name order (Ahmadi, Hassan, Karimi) - impossible to ask cleanly of one lumped 'Sara Ahmadi' cell.

```
    +------------+-------------+
    | given_name | family_name |
    +------------+-------------+
    | Sara       | Ahmadi      |
    | Lena       | Hassan      |
    | Omid       | Karimi      |
    +------------+-------------+
    (3 rows)
```

Split a composite WHEN you will search or sort by a part; keep it whole only if you truly never will. Then drop the demo - full_name stays as-is elsewhere:

```
    DROP TABLE member_split;
```

KEY IDEA: CREATE TABLE is your paper design becoming real; SELECT is you asking it a question; and the little words PRIMARY KEY, NOT NULL, and UNIQUE are rules the database then enforces for you, for free, forever. Everything else in this lesson is variations on these.

MAPS TO: primer Part 2 (entities and attributes) and companion Part 2 (the paper -> grid -> SQL costumes). Naming it the primer's way: the member TABLE is the entity TYPE, each ROW is one instance of it. The three rules map to primer Part 3 (keys and identity): card_no and member_id are each a CANDIDATE KEY (each one picks out a single row); we chose member_id as the PRIMARY KEY and kept card_no under a UNIQUE rule. Both member_id here and student-panel's UUID are SURROGATE keys (invented, meaningless handles), while card_no / student_code are NATURAL (a value people already use) - the "keep the useful natural value under a UNIQUE rule" move from companion confusion 3; samples PDF Lessons 1-2 (table/row) and 3-4 (keys).
student-panel difference: here member_id is a hand-typed INTEGER and card_no is our unique human code. student-panel uses a UUID primary key PLUS a unique human student_code - same two jobs (a stable surrogate id, and a unique human-facing code), different costume, exactly as companion confusion 3 warned.

PART 3 - FEEL THE PAIN (MAKE THE CLASSIC MISTAKES ON PURPOSE)

WHAT: build three tables the WRONG way and let them fail. WHY: the primer's normal-form rules sound abstract until a table refuses to answer a simple question. We will make it refuse. Everything in this part is BUILD then BREAK; the FIX for all of it lives in Part 4.

E2. REPEATING COLUMNS (phone1, phone2, phone3).

>> CONTEMPLATE: a member might have several phone numbers. The tempting fix is three columns. What happens when someone has a fourth? Or when you want "find the member with phone 555-0300" without knowing which column it is in?

-- BUILD (the wrong way) --

TRY IT:

```
CREATE TABLE member_bad_phones (
  member_id  INTEGER PRIMARY KEY,
  full_name  TEXT NOT NULL,
  phone1     TEXT,
  phone2     TEXT,
  phone3     TEXT
);

INSERT INTO member_bad_phones VALUES
  (1, 'Sara Ahmadi', '555-0100', '555-0101', NULL),
  (2, 'Omid Karimi', '555-0200', NULL, NULL);

SELECT * FROM member_bad_phones;
```

LOOK FOR: (your tool may show empty cells where this draws the word NULL)

```
+-----------+-------------+----------+----------+----------+
| member_id | full_name   | phone1   | phone2   | phone3   |
+-----------+-------------+----------+----------+----------+
| 1         | Sara Ahmadi | 555-0100 | 555-0101 | NULL     |
| 2         | Omid Karimi | 555-0200 | NULL     | NULL     |
+-----------+-------------+----------+----------+----------+
(2 rows)
```

Already it looks wasteful: Omid's row carries two empty slots, and nobody can have a fourth phone without you editing the table's shape.

-- BREAK (ask it an honest question) -------------------------------------

TRY IT - "which member has phone 555-0101?" You are forced to name every phone column by hand:

```
SELECT full_name FROM member_bad_phones
WHERE phone1 = '555-0101' OR phone2 = '555-0101' OR phone3 = '555-0101';
```

LOOK FOR: it works, but only because you spelled out all three columns.

```
+-------------+
| full_name   |
+-------------+
| Sara Ahmadi |
+-------------+
(1 row)
```

TRY IT - now count Sara's phones. There is no clean way; you must test every column by hand:

```
SELECT
    (CASE WHEN phone1 IS NOT NULL THEN 1 ELSE 0 END)
  + (CASE WHEN phone2 IS NOT NULL THEN 1 ELSE 0 END)
  + (CASE WHEN phone3 IS NOT NULL THEN 1 ELSE 0 END) AS phone_count
FROM member_bad_phones WHERE member_id = 1;
```

LOOK FOR:

```
+-------------+
```

```
    | phone_count |
    +-------------+
    | 2           |
    +-------------+
    (1 row)
```

FEEL IT: every question about phones makes you recite phone1, phone2, phone3. Add phone4 tomorrow and this query - and every query like it - is silently WRONG until you remember to edit it. The design fights you, and it fights you in a way the database cannot warn you about.

>> PITFALL: "I'll just add more columns when I need them." Columns are the SHAPE of your data; needing to add a column just to store one more of the same thing is the signal that the thing wants its own TABLE (we fix this in E5).

MAPS TO: primer Part 6 (1NF, no repeating groups) and primer Part 2.2 (a multi-valued attribute cannot be columns); samples PDF Lesson 14 (the phone1/2/3 break); companion confusion 7. student-panel avoids this entirely - a student's many values live in their own rows, never in numbered columns.
E3. A LIST STUFFED INTO ONE CELL.

>> CONTEMPLATE: the other tempting shortcut is one column holding "555-0100, 555-0101". Looks tidy. What breaks?

-- BUILD (the wrong way) --

TRY IT:

```
    CREATE TABLE member_bad_list (
      member_id  INTEGER PRIMARY KEY,
      full_name  TEXT NOT NULL,
      phones     TEXT
    );

    INSERT INTO member_bad_list VALUES
      (1, 'Sara Ahmadi', '555-0100, 555-0101'),
      (2, 'Omid Karimi', '555-0200');

    SELECT * FROM member_bad_list;
```

LOOK FOR:

```
    +-----------+-------------+---------------------+
    | member_id | full_name   | phones              |
    +-----------+-------------+---------------------+
    | 1         | Sara Ahmadi | 555-0100, 555-0101  |
    | 2         | Omid Karimi | 555-0200            |
    +-----------+-------------+---------------------+
    (2 rows)
```

-- BREAK (the list hides the data) ---------------------------------------

TRY IT - the honest question "who has phone 555-0100?":

```
    SELECT full_name FROM member_bad_list WHERE phones = '555-0100';
```

LOOK FOR: NOTHING - zero rows.

```
    +-------------+
    | full_name   |
    +-------------+
    +-------------+
    (0 rows)
```

Sara really does have 555-0100, but her cell holds the STRING '555-0100, 555-0101', which is not EQUAL to '555-0100'. The list hid her from an exact search.
TRY IT - the usual "fix" people reach for is pattern-matching with LIKE:

```
SELECT full_name FROM member_bad_list WHERE phones LIKE '%555-0100%';
```

LOOK FOR: Sara appears now -

```
+-------------+
| full_name   |
+-------------+
| Sara Ahmadi |
+-------------+
(1 row)
```

but this is a patch over a crack, not a repair. To see it crack, we switch to a second list - a student's SUBJECTS - and ask the primer's classic 1NF question, "who takes Science?"

TRY IT:

```
CREATE TABLE student_bad_subjects (
  student_id  INTEGER PRIMARY KEY,
  full_name   TEXT NOT NULL,
  subjects    TEXT
);

INSERT INTO student_bad_subjects VALUES
  (1, 'Sara', 'Social Science'),
  (2, 'Omid', 'Math, Science'),
  (3, 'Lena', 'History, Science');

SELECT * FROM student_bad_subjects;
```

LOOK FOR:

```
+------------+-----------+------------------+
| student_id | full_name | subjects         |
+------------+-----------+------------------+
| 1          | Sara      | Social Science   |
| 2          | Omid      | Math, Science    |
| 3          | Lena      | History, Science |
+------------+-----------+------------------+
(3 rows)
```

The true answer to "who takes Science?" is Omid and Lena. Sara takes SOCIAL Science, which is a different subject. Watch both tools fail that question.
TRY IT - exact match UNDER-answers (misses the two who really take Science):

```
SELECT full_name FROM student_bad_subjects WHERE subjects = 'Science';
```

LOOK FOR: zero rows - nobody's cell is the single word 'Science'.

```
+-----------+
| full_name |
+-----------+
+-----------+
(0 rows)
```

TRY IT - LIKE OVER-answers (drags in Sara, who takes Social Science):

```
    SELECT full_name FROM student_bad_subjects WHERE subjects LIKE '%Science%';

LOOK FOR: all three - but Sara does NOT take Science.

    +-----------+
    | full_name |
    +-----------+
    | Sara      |
    | Omid      |
    | Lena      |
    +-----------+
    (3 rows)

FEEL IT: '=' gives you too few, LIKE gives you too many, and neither can give
you the right two, because the word "Science" is buried inside "Social Science"
and the database has no way to know you meant a whole item in a list. Counting,
sorting, guaranteeing no duplicates, pointing a foreign key at one subject - all
impossible while the subjects live mashed into a single cell.

KEY IDEA: one value per cell. A list in a cell is data you cannot ask questions
about - which defeats the whole point of a database. The fix is never a cleverer
WHERE; it is a child table with one row per value (phones in E5, subjects in the
enrolment bridge of E7).

>> PITFALL: a comma-list is really the poor cousin of a flexible JSON column -
both stuff many facts into one opaque cell. The primer's judgement (Part 9, "a
JSON / flexible column vs proper columns") is the same lesson you just felt by
hand: anything you will filter, sort, count, or point a foreign key at deserves
its own column or row; only genuinely open-ended, never-queried extras belong in
a free-form blob.

MAPS TO: primer Part 6 (1NF, atomic values); samples PDF Lesson 14; companion
confusion 7 ("who takes science?" is the exact example named there). This is
the pain the companion's confusion 7 warned you about, now felt in your hands.
student-panel avoids it: every multi-valued thing (a student's scores, a
group's members) is its own table of rows, never a comma-list in a column.
E4. ONE GIANT WIDE TABLE (the redundancy / update anomaly).

>> CONTEMPLATE: imagine tracking library loans in a SINGLE table that repeats
the member's name and the book's title on every loan row. What goes wrong the
day a member changes their name?

-- BUILD (the wrong way) ------------------------------------------------

TRY IT:

    CREATE TABLE loan_wide (
      loan_id      INTEGER PRIMARY KEY,
      member_name  TEXT,
      book_title   TEXT,
      borrowed_on  TEXT
    );

    INSERT INTO loan_wide VALUES
      (1, 'Sara Ahmadi', 'The Blue Door', '2026-09-10'),
      (2, 'Sara Ahmadi', 'Quiet Rivers',  '2026-09-12'),
      (3, 'Omid Karimi', 'The Blue Door', '2026-09-13');

    SELECT * FROM loan_wide;

LOOK FOR: Sara's name sits on two rows; 'The Blue Door' sits on two rows.

    +---------+-------------+---------------+-------------+
```

```
    | loan_id | member_name | book_title    | borrowed_on |
    +---------+-------------+---------------+-------------+
    | 1       | Sara Ahmadi | The Blue Door | 2026-09-10  |
    | 2       | Sara Ahmadi | Quiet Rivers  | 2026-09-12  |
    | 3       | Omid Karimi | The Blue Door | 2026-09-13  |
    +---------+-------------+---------------+-------------+
    (3 rows)
```

Every repeat is a copy that can later disagree with its twin. Watch it happen.

```
-- BREAK 1: the UPDATE anomaly ----------------------------------------------
```

TRY IT - Sara married and is now 'Sara Noori'. You update the row you happen to be looking at:

```
    UPDATE loan_wide SET member_name = 'Sara Noori' WHERE loan_id = 1;

    SELECT * FROM loan_wide;
```

LOOK FOR / FEEL IT: loan 1 says 'Sara Noori' but loan 2 STILL says 'Sara Ahmadi'. The same person now wears two names in one table.

```
    +---------+-------------+---------------+-------------+
    | loan_id | member_name | book_title    | borrowed_on |
    +---------+-------------+---------------+-------------+
    | 1       | Sara Noori  | The Blue Door | 2026-09-10  |
    | 2       | Sara Ahmadi | Quiet Rivers  | 2026-09-12  |
    | 3       | Omid Karimi | The Blue Door | 2026-09-13  |
    +---------+-------------+---------------+-------------+
    (3 rows)
```

This is the UPDATE ANOMALY the primer names: a fact duplicated across rows drifts apart the moment one copy changes. Nobody lied; the SHAPE of the table made the contradiction possible.

```
-- BREAK 2: the DELETE anomaly ----------------------------------------------
```

TRY IT - Omid returns his only book, so you delete that loan:

```
    DELETE FROM loan_wide WHERE loan_id = 3;

    SELECT member_name FROM loan_wide;
```

LOOK FOR / FEEL IT: Omid has vanished from the database entirely - the only place his name lived was that loan row.

```
    +-------------+
    | member_name |
    +-------------+
    | Sara Noori  |
    | Sara Ahmadi |
    +-------------+
    (2 rows)
```

```
-- BREAK 3: the INSERT anomaly (a contemplation, not a query) ---------------
```

>> CONTEMPLATE: suppose a brand-new member, 'Lena Hassan', joins but has not borrowed anything yet. Where in loan_wide do you put her? There is no row to put her on, because this table only has room for a person WHEN they borrow a book. You would have to invent a fake loan with empty book columns - a lie in the data. That is the INSERT anomaly: the table cannot record a true fact (a member exists) because it tangled that fact together with another (a loan).

The book title is duplicated too, with the same drift risk: rename 'The Blue Door' on one loan and the other loan still shows the old title.

So three different pains all live in this one wide table:
  - UPDATE anomaly - you saw Sara split into two names.
  - DELETE anomaly - you saw Omid disappear when his last loan went.
  - INSERT anomaly - you cannot even record Lena until she borrows something.

KEY IDEA: store each fact ONCE, in the table it belongs to, and POINT at it. A member's name belongs in a member table; a loan should only POINT at the member, not re-type their name. When each fact lives in one place, there is no second copy to drift, nothing to lose when a loan is deleted, and a member can exist with zero loans.

MAPS TO: primer Part 6 (the three anomalies; 2NF and 3NF); samples PDF Lesson 16; primer Part 1 ("why not just one big spreadsheet?" - this is that very spreadsheet). student-panel keeps each fact once: a score row points at a student and a parameter, it does not copy the student's name - which is why renaming a student there changes it everywhere at once, with no drift. In E6 you will build loans the right way and watch all three anomalies disappear.

PART 4 - FIX IT (TURN THE RULES INTO TABLES)

Now we repair every pain from Part 3 using the exact moves from the primer. WHAT: four fixes - own-table-for-many-values, split-and-point (with the three ON DELETE rules), a bridge, a tree. WHY: these four moves cover the large majority of real designs, student-panel included. Each exercise now walks the full loop: BUILD the good shape, BREAK it to prove the new rules bite, FIX / extend, then TEST with a real question.

E5. FIX THE PHONES: give the repeating thing its OWN table (the "one move").

>> CONTEMPLATE: a member HAS MANY phones; a phone belongs to ONE member. That is one-to-many. From the companion (confusion 4), where does the foreign key go? (On the many side - the phone carries member_id, not the other way round.)

-- BUILD (the right way) --

TRY IT:

```
CREATE TABLE member2 (
  member_id  INTEGER PRIMARY KEY,
  full_name  TEXT NOT NULL
);

CREATE TABLE phone (
  phone_id   INTEGER PRIMARY KEY,
  member_id  INTEGER NOT NULL REFERENCES member2(member_id),
  number     TEXT NOT NULL
);

INSERT INTO member2 VALUES
  (1, 'Sara Ahmadi'),
  (2, 'Omid Karimi'),
  (3, 'Lena Hassan');

INSERT INTO phone VALUES
  (10, 1, '555-0100'),
  (11, 1, '555-0101'),
  (12, 2, '555-0200'),
  (13, 3, '555-0300');

SELECT * FROM phone;
```

LOOK FOR: one row per phone - each pointing at its member by member_id.

```
+----------+-----------+----------+
| phone_id | member_id | number   |
+----------+-----------+----------+
| 10       | 1         | 555-0100 |
| 11       | 1         | 555-0101 |
| 12       | 2         | 555-0200 |
| 13       | 3         | 555-0300 |
+----------+-----------+----------+
(4 rows)
```

TRY IT - the question that was painful in E2/E3 is now clean, and works no matter how many phones anyone has:

```
SELECT m.full_name
FROM member2 m JOIN phone p ON p.member_id = m.member_id
WHERE p.number = '555-0101';
```

LOOK FOR: Sara. One row, no OR-chain, no hidden list.

```
+-------------+
| full_name   |
+-------------+
| Sara Ahmadi |
+-------------+
(1 row)
```

TRY IT - and the question E2 only daydreamed about ("find the member with phone 555-0300") is now a real answer:

```
SELECT m.full_name
FROM member2 m JOIN phone p ON p.member_id = m.member_id
WHERE p.number = '555-0300';
```

LOOK FOR: Lena.

```
+-------------+
| full_name   |
+-------------+
| Lena Hassan |
+-------------+
(1 row)
```

TRY IT - counting phones, which took an ugly CASE pile-up in E2, is now one clean line per member:

```
SELECT m.full_name, COUNT(p.phone_id) AS phones
FROM member2 m JOIN phone p ON p.member_id = m.member_id
GROUP BY m.full_name
ORDER BY m.full_name;
```

LOOK FOR:

```
+-------------+--------+
| full_name   | phones |
+-------------+--------+
| Lena Hassan | 1      |
| Omid Karimi | 1      |
| Sara Ahmadi | 2      |
+-------------+--------+
(3 rows)
```

Notice what we did NOT do: we never store a "phone_count" column, we COMPUTE it with COUNT whenever we ask. That is the primer's "derive, don't store" rule

(primer Part 2.2 and Part 9) - a value you can compute from other rows should be recomputed on demand, so it can never drift out of date the way a stored count would the instant a phone is added or removed. (The one deliberate exception - a value you FREEZE on purpose, like the recorded grade in E7 - is history, not derivation, and is allowed to stand still.)

-- BREAK (the foreign key now refuses an orphan) -------------------------

TRY IT - try to give a phone to a member who does not exist:

```
INSERT INTO phone VALUES (99, 999, '555-9999');
```

LOOK FOR: refused - there is no member 999 for this phone to belong to.

```
ERROR (Option A, Postgres / PGlite):
  ERROR:  insert or update on table "phone" violates foreign key
  constraint "phone_member_id_fkey"
  DETAIL:  Key (member_id)=(999) is not present in table "member2".
ERROR (Option B, SQLite):
  Error: FOREIGN KEY constraint failed
```

(SQLite users: if this INSERT is wrongly ACCEPTED, your foreign keys are off - run PRAGMA foreign_keys = ON; from Part 0 and try again.)
-- TEST (growth needs no redesign) ---------------------------------------

TRY IT - give Sara a FOURTH phone. In E2 this needed a new column; here it is just one more row, with zero change to the table's shape:

```
INSERT INTO phone VALUES (14, 1, '555-0109');

SELECT COUNT(*) AS sara_phones FROM phone WHERE member_id = 1;
```

LOOK FOR:

```
+-------------+
| sara_phones |
+-------------+
| 3           |
+-------------+
(1 row)
```

TRY IT - now remove that extra phone so our running library stays tidy for the later exercises (and notice deleting a CHILD row is always allowed - it breaks no promise):

```
DELETE FROM phone WHERE phone_id = 14;
```

We are back to four phone rows. A member can now have one, four, or forty phones with no redesign at all.

KEY IDEA: "many of the same thing" is never more columns and never a list in a cell - it is a child table with a foreign key pointing back at the parent. The crow's foot (many) sits on the child, which is exactly where member_id lives.

MAPS TO: primer Part 2.2 (a multi-valued attribute becomes its own rows) and Part 5 (the patterns); samples PDF Lessons 3 and 14; companion confusion 4 (the FK goes on the many side). REFERENCES is the foreign key made real.
student-panel: a student's many values live in child tables exactly like this - same shape, just UUID pointers instead of our INTEGER member_id.

E6. FIX THE WIDE LOAN TABLE: split, point, and meet REFERENTIAL INTEGRITY.

>> CONTEMPLATE: the wide table (E4) mixed three things - members, books, and

loans. Separate them. A loan POINTS at one member and one book; it stops
copying their names. (We reuse member2 from E5 as the member table.)

-- BUILD (the right way) --

TRY IT:

```
CREATE TABLE book (
  book_id  INTEGER PRIMARY KEY,
  title    TEXT NOT NULL
);

CREATE TABLE loan (
  loan_id      INTEGER PRIMARY KEY,
  member_id    INTEGER NOT NULL REFERENCES member2(member_id),
  book_id      INTEGER NOT NULL REFERENCES book(book_id),
  borrowed_on  TEXT NOT NULL
);

INSERT INTO book VALUES
  (100, 'The Blue Door'),
  (101, 'Quiet Rivers'),
  (102, 'Open Sky');

INSERT INTO loan VALUES
  (1, 1, 100, '2026-09-10'),
  (2, 1, 101, '2026-09-12'),
  (3, 2, 100, '2026-09-13');

SELECT l.loan_id, m.full_name, b.title, l.borrowed_on
FROM loan l
JOIN member2 m ON m.member_id = l.member_id
JOIN book   b ON b.book_id   = l.book_id;
```

LOOK FOR: the loan table stores no names at all; the JOIN fetches them live.

```
+---------+-------------+---------------+-------------+
| loan_id | full_name   | title         | borrowed_on |
+---------+-------------+---------------+-------------+
| 1       | Sara Ahmadi | The Blue Door | 2026-09-10  |
| 2       | Sara Ahmadi | Quiet Rivers  | 2026-09-12  |
| 3       | Omid Karimi | The Blue Door | 2026-09-13  |
+---------+-------------+---------------+-------------+
(3 rows)
```

-- The payoff of splitting: fix a name in ONE place ----------------------

Remember the wide table (E4), where Sara's name sat on every loan row, so a
rename meant hunting down every copy. Now her name lives in exactly one row
of member2. Change it once:

TRY IT:

```
UPDATE member2 SET full_name = 'Sara Noori' WHERE member_id = 1;

SELECT l.loan_id, m.full_name, b.title, l.borrowed_on
FROM loan l
JOIN member2 m ON m.member_id = l.member_id
JOIN book   b ON b.book_id   = l.book_id;
```

LOOK FOR: both of Sara's loans now read 'Sara Noori', though you touched one
row. The loans never stored her name - they borrow it through the FK.

```
+---------+-------------+---------------+-------------+
```

```
    | loan_id | full_name   | title         | borrowed_on |
    +---------+-------------+---------------+-------------+
    | 1       | Sara Noori  | The Blue Door | 2026-09-10  |
    | 2       | Sara Noori  | Quiet Rivers  | 2026-09-12  |
    | 3       | Omid Karimi | The Blue Door | 2026-09-13  |
    +---------+-------------+---------------+-------------+
    (3 rows)

KEY IDEA: one fact, one place. Store each fact once; let everything else
point at it. That is the whole reason we split the wide table.
-- BREAK (the FK refuses an orphan) ---------------------------------------

>> CONTEMPLATE: what if a loan points at a member who does not exist?
Member 999 was never inserted. Try to lend a book to them.

TRY IT:

    INSERT INTO loan VALUES (4, 999, 100, '2026-09-14');

This FAILS. A foreign key only lets you point at a row that is really there.
The database refuses to create a loan with no member behind it:

  Option A - Postgres / PGlite:

    +-------------------------------------------------------------------------+
    | ERROR:  insert or update on table "loan" violates foreign key           |
    |         constraint "loan_member_id_fkey"                                |
    | DETAIL:  Key (member_id)=(999) is not present in table "member2".       |
    +-------------------------------------------------------------------------+

  Option B - SQLite (with PRAGMA foreign_keys = ON):

    +-------------------------------------------------------------------------+
    | Error: FOREIGN KEY constraint failed                                    |
    +-------------------------------------------------------------------------+

KEY IDEA: this is REFERENTIAL INTEGRITY. The FK is a promise the database
keeps for you: every member_id in loan really exists in member2. You cannot
end up with a loan that belongs to nobody.

>> PITFALL: in SQLite the check above only happens if you have run
`PRAGMA foreign_keys = ON;` first. Without it, SQLite accepts the orphan
silently and the promise is broken. Postgres and PGlite always enforce it.
-- The OTHER side of integrity: deleting a PARENT -------------------------

The orphan test blocked a bad CHILD. Now the opposite question: what happens
when you delete a PARENT that children already point at? That is where the
ON DELETE rules live. Our loan table above wrote no ON DELETE clause, so it
uses the DEFAULT, which is to BLOCK the delete.

First, the easy case - a parent nobody points at can always be deleted.
'Open Sky' (book 102) has no loans:

TRY IT:

    DELETE FROM book WHERE book_id = 102;   -- allowed: no loan points here
    INSERT INTO book VALUES (102, 'Open Sky');  -- put it back for later parts

LOOK FOR: the DELETE succeeds (it reports 1 row removed), because nothing
referenced it. We immediately re-insert 'Open Sky' so Part 5 can still count
it as a never-borrowed book.

Now the hard case - 'The Blue Door' (book 100) is referenced by loans 1 and 3:
```

TRY IT:

```
    DELETE FROM book WHERE book_id = 100;
```

This FAILS. With no ON DELETE rule, the database protects the children by
refusing to orphan them:

  Option A - Postgres / PGlite:

```
    +-------------------------------------------------------------------+
    | ERROR:  update or delete on table "book" violates foreign key     |
    |         constraint "loan_book_id_fkey" on table "loan"            |
    | DETAIL:  Key (book_id)=(100) is still referenced from table "loan".  |
    +-------------------------------------------------------------------+
```

  Option B - SQLite (with PRAGMA foreign_keys = ON):

```
    +-------------------------------------------------------------------+
    | Error: FOREIGN KEY constraint failed                              |
    +-------------------------------------------------------------------+
```

KEY IDEA, said plainly: with no ON DELETE clause, you cannot delete a parent
while any child still points at it. You must remove or re-point the children
first. This default has a name: NO ACTION (RESTRICT behaves the same way).
-- The three ON DELETE rules, each on its own tiny pair of tables ---------

You get to CHOOSE what a parent-delete does to the children. There are three
rules worth knowing. We will show each on its own small pair of tables so one
rule never interferes with another. Say each one as a plain sentence first,
then watch it act.

(1) ON DELETE RESTRICT - "you may not delete a parent that has children."
    This is the same protective behaviour as the default, written out loud.

TRY IT:

```
    CREATE TABLE member_r (
      member_id  INTEGER PRIMARY KEY,
      full_name  TEXT NOT NULL
    );
    CREATE TABLE loan_r (
      loan_id      INTEGER PRIMARY KEY,
      member_id    INTEGER NOT NULL REFERENCES member_r(member_id) ON DELETE RESTRICT,
      borrowed_on  TEXT NOT NULL
    );
    INSERT INTO member_r VALUES (1, 'Sara Ahmadi'), (2, 'Omid Karimi');
    INSERT INTO loan_r   VALUES (1, 1, '2026-09-10'), (2, 2, '2026-09-13');

    DELETE FROM member_r WHERE member_id = 1;   -- Sara has loan 1
```

This FAILS - loan 1 still points at Sara:

  Option A - Postgres / PGlite:

```
    +-------------------------------------------------------------------+
    | ERROR:  update or delete on table "member_r" violates foreign key    |
    |         constraint "loan_r_member_id_fkey" on table "loan_r"         |
    | DETAIL:  Key (member_id)=(1) is still referenced from table          |
    |          "loan_r".                                                |
    +-------------------------------------------------------------------+
```

  Option B - SQLite (with PRAGMA foreign_keys = ON):

```
    +-------------------------------------------------------------------+
    | Error: FOREIGN KEY constraint failed                              |
    +-------------------------------------------------------------------+

LOOK FOR: nothing was deleted. RESTRICT guards the parent until the children
are gone.
(2) ON DELETE CASCADE - "delete a parent and its children go with it."
    Good when a child cannot exist on its own, like a phone with no owner.

TRY IT:

    CREATE TABLE member_c (
      member_id  INTEGER PRIMARY KEY,
      full_name  TEXT NOT NULL
    );
    CREATE TABLE phone_c (
      phone_id   INTEGER PRIMARY KEY,
      member_id  INTEGER NOT NULL REFERENCES member_c(member_id) ON DELETE CASCADE,
      number     TEXT NOT NULL
    );
    INSERT INTO member_c VALUES (1, 'Sara Ahmadi'), (2, 'Omid Karimi');
    INSERT INTO phone_c  VALUES (10, 1, '555-0100'), (11, 1, '555-0101'), (12, 2, '555-0200');

    DELETE FROM member_c WHERE member_id = 1;   -- Sara owns phones 10 and 11
    SELECT * FROM phone_c ORDER BY phone_id;

LOOK FOR: deleting Sara also deleted BOTH her phones. Only Omid's remains.
You ran one DELETE; the database cleaned up the children for you.

    +----------+-----------+----------+
    | phone_id | member_id | number   |
    +----------+-----------+----------+
    | 12       | 2         | 555-0200 |
    +----------+-----------+----------+
    (1 row)

>> PITFALL: CASCADE is powerful and quiet. Deleting one parent can erase a
whole subtree of children with no second warning. Reach for it only when the
children are truly meaningless without the parent.
(3) ON DELETE SET NULL - "delete a parent and the children keep living, but
    their pointer is emptied to NULL." Good when the link is optional - a
    loan can remember it was handled, even after that staff member leaves.
    NOTE: the FK column must be NULLable for this to work (no NOT NULL).

TRY IT:

    CREATE TABLE staff_s (
      staff_id  INTEGER PRIMARY KEY,
      name      TEXT NOT NULL
    );
    CREATE TABLE loan_s (
      loan_id      INTEGER PRIMARY KEY,
      borrowed_on  TEXT NOT NULL,
      handled_by   INTEGER REFERENCES staff_s(staff_id) ON DELETE SET NULL
    );
    INSERT INTO staff_s VALUES (1, 'Librarian Ada'), (2, 'Librarian Ben');
    INSERT INTO loan_s  VALUES (1, '2026-09-10', 1), (2, '2026-09-13', 2), (3, '2026-09-14', NULL);

    DELETE FROM staff_s WHERE staff_id = 1;   -- Ada handled loan 1
    SELECT * FROM loan_s ORDER BY loan_id;

LOOK FOR: loan 1 is still there - the loan really happened - but handled_by
```

is now NULL because Ada is gone. Loan 2 (Ben) is untouched. Loan 3 was
already NULL.

```
    +---------+-------------+------------+
    | loan_id | borrowed_on | handled_by |
    +---------+-------------+------------+
    | 1       | 2026-09-10  | NULL       |
    | 2       | 2026-09-13  | 2          |
    | 3       | 2026-09-14  | NULL       |
    +---------+-------------+------------+
    (3 rows)
```

KEY IDEA: pick the rule by asking "if the parent disappears, what SHOULD
happen to the children?" - block it (RESTRICT), take them along (CASCADE),
or keep them but forget the link (SET NULL).
-- TEST (the running data survived the experiments) -----------------------

The three pairs above were scratch tables. Confirm the real running data is
untouched: three loans, three books (including Open Sky), Sara still renamed.

TRY IT:

```
    SELECT (SELECT COUNT(*) FROM loan)    AS loans,
           (SELECT COUNT(*) FROM book)    AS books,
           (SELECT full_name FROM member2 WHERE member_id = 1) AS member_1;
```

```
    +-------+-------+------------+
    | loans | books | member_1   |
    +-------+-------+------------+
    | 3     | 3     | Sara Noori |
    +-------+-------+------------+
    (1 row)
```

Now drop the scratch tables so they do not clutter the later exercises
(children first, then parents - the same RESTRICT rule applies to dropping):

```
    DROP TABLE loan_r;    DROP TABLE member_r;
    DROP TABLE phone_c;   DROP TABLE member_c;
    DROP TABLE loan_s;    DROP TABLE staff_s;
```

MAPS TO: this is primer Part 5 (one-to-many: the FK lives on the many side,
here loan) and primer Part 9 (referential integrity and the ON DELETE rules).
It is companion confusion 6 ("which table gets the foreign key?" - the child)
and confusion 10 (what DELETE really does). In the samples PDF it is Lessons
6-8 (relationships and joins) and Lesson 16 (delete behaviour).

>> Two deliberate differences from student-panel, at the PRINCIPLE level:
  - IDs: here loans use hand-typed INTEGER keys so you can read them. The
    student-panel project uses UUIDs plus a human-friendly student_code. Same
    idea (one unique key per row), different shape.
  - DELETE: here we HARD-delete rows so you can watch CASCADE/RESTRICT/SET
    NULL act. The student-panel project SOFT-deletes - it sets an archived_at
    timestamp and hides the row instead of removing it, so history and past
    scores are never lost. Same goal (don't leave dangling references),
    gentler mechanism.
  - MODEL: real libraries track each physical COPY of a book (so two people
    can be told "all copies are out"). We collapsed book+copy into one `book`
    table on purpose, to keep the running example small. Note the
    simplification; do not treat our `book` as a full catalogue design.

E7. MANY-TO-MANY: STUDENTS, COURSES, AND A BRIDGE TABLE.

>> CONTEMPLATE: a student takes many courses. A course holds many students.

This is many-to-many, and it is the one shape where a plain foreign key is
not enough. If you put a course_id on student, a student could hold only one
course. If you put a student_id on course, a course could hold only one
student. Neither is true. You need a THIRD table that sits between them -
a bridge - where each row is one pairing: "this student is in this course."

This also finishes the job we started in E3. There we jammed subjects into a
comma-list ('Math, Science') and found we could not ask "who takes science?"
The bridge is the real fix.

-- BUILD (two sides and a bridge) --

TRY IT:

```
CREATE TABLE student (
  student_id  INTEGER PRIMARY KEY,
  full_name   TEXT NOT NULL
);

CREATE TABLE course (
  course_id  INTEGER PRIMARY KEY,
  title      TEXT NOT NULL
);

CREATE TABLE enrolment (
  student_id  INTEGER NOT NULL REFERENCES student(student_id),
  course_id   INTEGER NOT NULL REFERENCES course(course_id),
  grade       TEXT,
  PRIMARY KEY (student_id, course_id)
);

INSERT INTO student VALUES
  (1, 'Sara Ahmadi'),
  (2, 'Omid Karimi'),
  (3, 'Lena Hassan');

INSERT INTO course VALUES
  (10, 'Algebra'),
  (11, 'Science'),
  (12, 'History');

INSERT INTO enrolment VALUES
  (1, 10, 'A'),
  (1, 11, 'B'),
  (2, 10, 'B'),
  (3, 11, 'A'),
  (3, 12, 'C');

SELECT s.full_name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
ORDER BY s.full_name, c.title;
```

LOOK FOR: each row is one student-in-one-course pairing, and it carries a
fact of its own - the grade. The grade belongs to the PAIRING, not to the
student alone or the course alone.

```
+-------------+---------+-------+
| full_name   | title   | grade |
+-------------+---------+-------+
| Lena Hassan | History | C     |
| Lena Hassan | Science | A     |
```

```
    | Omid Karimi | Algebra | B     |
    | Sara Ahmadi | Algebra | A     |
    | Sara Ahmadi | Science | B     |
    +-------------+---------+-------+
    (5 rows)

KEY IDEA: the bridge table's primary key is the PAIR of foreign keys -
PRIMARY KEY (student_id, course_id). That is a COMPOSITE key: two columns
that are unique together. It also says a student can be in a given course
only once.

-- A WRONG TURN WORTH TAKING: 2NF (keep part-key facts off the bridge) -----

>> CONTEMPLATE: the bridge's key is the PAIR (student_id, course_id), and grade
needs BOTH halves to mean anything. But what if we ALSO parked the student's
NAME on the bridge? The name depends on student_id ALONE - only HALF the key.
That is exactly what Second Normal Form forbids. Feel it:

TRY IT (the wrong shape - the name is copied onto every enrolment of a student):

    CREATE TABLE enrolment_bad (
      student_id   INTEGER NOT NULL,
      course_id    INTEGER NOT NULL,
      student_name TEXT NOT NULL,
      grade        TEXT,
      PRIMARY KEY (student_id, course_id)
    );

    INSERT INTO enrolment_bad VALUES
      (1, 10, 'Sara Ahmadi', 'A'),
      (1, 11, 'Sara Ahmadi', 'B'),
      (3, 11, 'Lena Hassan', 'A');

    SELECT * FROM enrolment_bad ORDER BY student_id, course_id;

LOOK FOR: 'Sara Ahmadi' is written on BOTH her rows. Change it in one and the
two disagree about one real fact - the same drift you felt in E4, now caused by
a PART-of-the-key dependency.

    +------------+-----------+--------------+-------+
    | student_id | course_id | student_name | grade |
    +------------+-----------+--------------+-------+
    | 1          | 10        | Sara Ahmadi  | A     |
    | 1          | 11        | Sara Ahmadi  | B     |
    | 3          | 11        | Lena Hassan  | A     |
    +------------+-----------+--------------+-------+
    (3 rows)

KEY IDEA (2NF): a non-key fact must depend on the WHOLE key, not half of it.
student_name depends on student_id alone, so it belongs in the student table
(where E7 correctly keeps it) - only grade, which needs BOTH halves, stays on
the bridge. Drop the bad table:

    DROP TABLE enrolment_bad;

-- BREAK (two ways the bridge protects you) --------------------------------

First, a duplicate pairing. Sara (1) is already in Algebra (10). Try to add
her again:

TRY IT:

    INSERT INTO enrolment VALUES (1, 10, 'A');
```

This FAILS. The composite primary key says each (student, course) pair is
unique, so the same enrolment cannot exist twice:

  Option A - Postgres / PGlite:

```
+-------------------------------------------------------------------+
| ERROR:  duplicate key value violates unique constraint            |
|         "enrolment_pkey"                                          |
| DETAIL:  Key (student_id, course_id)=(1, 10) already exists.      |
+-------------------------------------------------------------------+
```

  Option B - SQLite:

```
+-------------------------------------------------------------------+
| Error: UNIQUE constraint failed: enrolment.student_id,            |
|        enrolment.course_id                                        |
+-------------------------------------------------------------------+
```

Now an orphan pairing. Course 99 does not exist:

TRY IT:

```
INSERT INTO enrolment VALUES (1, 99, 'A');
```

This FAILS too. Each side of the bridge is a foreign key, so you cannot pair
a student with a course that is not there:

  Option A - Postgres / PGlite:

```
+-------------------------------------------------------------------+
| ERROR:  insert or update on table "enrolment" violates foreign key|
|         constraint "enrolment_course_id_fkey"                     |
| DETAIL:  Key (course_id)=(99) is not present in table "course".   |
+-------------------------------------------------------------------+
```

  Option B - SQLite (with PRAGMA foreign_keys = ON):

```
+-------------------------------------------------------------------+
| Error: FOREIGN KEY constraint failed                              |
+-------------------------------------------------------------------+
```

KEY IDEA: a bridge table enforces two promises at once - no duplicate pairing
(its composite PK) and no pairing with a missing side (its two FKs).
-- A VARIATION: when the "duplicate" is really allowed (retakes) -----------

>> CONTEMPLATE: what if Sara retakes Algebra in a later term? With the key
(student_id, course_id), the second enrolment looks like a forbidden
duplicate. The fix is to widen the key: add the term, so the pairing becomes
"this student, this course, THIS term."

TRY IT:

```
CREATE TABLE enrolment2 (
  student_id  INTEGER NOT NULL REFERENCES student(student_id),
  course_id   INTEGER NOT NULL REFERENCES course(course_id),
  term        TEXT NOT NULL,
  grade       TEXT,
  PRIMARY KEY (student_id, course_id, term)
);

INSERT INTO enrolment2 VALUES
  (1, 10, '2026-Fall',   'A'),
```

```
      (1, 10, '2026-Spring', 'C');

    SELECT s.full_name, c.title, e.term, e.grade
    FROM enrolment2 e
    JOIN student s ON s.student_id = e.student_id
    JOIN course  c ON c.course_id  = e.course_id
    ORDER BY e.term;

LOOK FOR: Sara is now in Algebra TWICE, and both are legal, because the term
makes each pairing unique.

    +-------------+---------+-------------+-------+
    | full_name   | title   | term        | grade |
    +-------------+---------+-------------+-------+
    | Sara Ahmadi | Algebra | 2026-Fall   | A     |
    | Sara Ahmadi | Algebra | 2026-Spring | C     |
    +-------------+---------+-------------+-------+
    (2 rows)

But the SAME student, course, AND term together are still unique. Try to add
the Fall row again:

TRY IT:

    INSERT INTO enrolment2 VALUES (1, 10, '2026-Fall', 'A');

  Option A - Postgres / PGlite:

    +-----------------------------------------------------------------------+
    | ERROR:  duplicate key value violates unique constraint                |
    |         "enrolment2_pkey"                                             |
    | DETAIL:  Key (student_id, course_id, term)=(1, 10, 2026-Fall)         |
    |          already exists.                                              |
    +-----------------------------------------------------------------------+

  Option B - SQLite:

    +-----------------------------------------------------------------------+
    | Error: UNIQUE constraint failed: enrolment2.student_id,               |
    |        enrolment2.course_id, enrolment2.term                          |
    +-----------------------------------------------------------------------+

KEY IDEA: the primary key is a design choice about "what makes a row unique."
Add a column to the key when the real world says the old pairing can repeat.

Drop the variation table so the later parts use the simple enrolment:

    DROP TABLE enrolment2;

-- THE OTHER KEY CHOICE: a surrogate id ON the bridge (+ keep UNIQUE) -------

>> CONTEMPLATE: so far the bridge is keyed by the natural PAIR. The other honest
option is to give the bridge its OWN surrogate id (one tidy value for other
tables to point at) - BUT then nothing stops a duplicate pairing unless you ALSO
add a UNIQUE rule on (student_id, course_id). Watch what the UNIQUE buys back:

TRY IT:

    CREATE TABLE enrolment_srg (
      enrolment_id INTEGER PRIMARY KEY,
      student_id   INTEGER NOT NULL REFERENCES student(student_id),
      course_id    INTEGER NOT NULL REFERENCES course(course_id),
      grade        TEXT,
```

```
      UNIQUE (student_id, course_id)
    );

    INSERT INTO enrolment_srg VALUES (1, 1, 10, 'A');
    INSERT INTO enrolment_srg VALUES (2, 1, 10, 'B');   -- same pair again
```

LOOK FOR: the second insert is REFUSED - a different enrolment_id, but the UNIQUE on the pair still blocks "Sara in Algebra twice":

Option A - Postgres / PGlite:

```
    +-----------------------------------------------------------------------+
    | ERROR:  duplicate key value violates unique constraint                |
    |         "enrolment_srg_student_id_course_id_key"                      |
    | DETAIL:  Key (student_id, course_id)=(1, 10) already exists.          |
    +-----------------------------------------------------------------------+
```

Option B - SQLite:

```
    +-----------------------------------------------------------------------+
    | Error: UNIQUE constraint failed: enrolment_srg.student_id,            |
    |        enrolment_srg.course_id                                        |
    +-----------------------------------------------------------------------+
```

KEY IDEA: when a bridge (or any middle table) gets its own surrogate id, DECIDE whether the natural combination must be UNIQUE - here it must, or the duplicate sneaks back in through the side door. This is exactly how student-panel's scores table behaves: a surrogate id, PLUS the student x parameter x period combination kept unique. Drop the demo:

```
    DROP TABLE enrolment_srg;
```

-- TEST (close the E3 loop: now we CAN ask "who takes X?") -----------------

Back in E3, subjects lived in a comma-list and the question "who takes science?" had no answer in SQL. With the bridge, it is one line:

TRY IT:

```
    SELECT s.full_name
    FROM enrolment e
    JOIN student s ON s.student_id = e.student_id
    WHERE e.course_id = 11            -- 11 = Science
    ORDER BY s.full_name;
```

```
    +-------------+
    | full_name   |
    +-------------+
    | Lena Hassan |
    | Sara Ahmadi |
    +-------------+
    (2 rows)
```

And the same shape answers "who takes Algebra?" - just change the course_id:

TRY IT:

```
    SELECT s.full_name
    FROM enrolment e
    JOIN student s ON s.student_id = e.student_id
    WHERE e.course_id = 10            -- 10 = Algebra
    ORDER BY s.full_name;
```

```
+-------------+
| full_name   |
+-------------+
| Omid Karimi |
| Sara Ahmadi |
+-------------+
(2 rows)
```

LOOK FOR: the comma-list could not do this; the bridge does it with WHERE.
That is the whole reason E3 was a trap and E7 is the cure.

MAPS TO: this is primer Part 5.1 (the many-to-many bridge / junction table) and primer Part 3 (composite keys). It is companion confusion 5 (why a many-to-many needs a third table). In the samples PDF it is Lessons 9 (bridge tables) and 10-11 (composite keys and junctions). The grade-on-the-bridge idea - a fact that belongs to the pairing - is exactly how the student-panel project stores a SCORE: not on the student and not on the parameter, but on the row that joins a student to a parameter for a period. And that recorded grade (or score) is a FROZEN FACT in the primer's Part 9 sense: once written it is history and must not silently change if a parameter is edited later - which is why it is NOT the "derive, don't store" count of E5, and NOT lazy denormalisation, but a separate fact that merely resembles a live value.

E8. A SELF-REFERENCE TREE: GROUPS INSIDE GROUPS.

>> CONTEMPLATE: sometimes a thing contains more of the same thing. A group of students can hold sub-groups, which hold their own sub-groups. There is no second table here - a category simply POINTS at its own parent category, in the SAME table. The pointer is a foreign key from the table back to itself. The top of the tree (the root) has no parent, so its parent_id is NULL.

-- BUILD (one table that references itself) ------------------------------

TRY IT:

```
CREATE TABLE category (
  category_id  INTEGER PRIMARY KEY,
  name         TEXT NOT NULL,
  parent_id    INTEGER REFERENCES category(category_id)
);

INSERT INTO category VALUES
  (1, 'All Students',        NULL),   -- root: no parent
  (2, 'Grade 7',             1),      -- child of All Students
  (3, 'Grade 8',             1),
  (4, 'Grade 7 - Section A', 2),      -- child of Grade 7
  (5, 'Grade 7 - Section B', 2),
  (6, 'Grade 8 - Section A', 3);

SELECT category_id, name, parent_id FROM category ORDER BY category_id;
```

LOOK FOR: parent_id is just another column, but it holds a category_id from THIS table. Row 1 has NULL - it is the root.

```
+-------------+---------------------+-----------+
| category_id | name                | parent_id |
+-------------+---------------------+-----------+
| 1           | All Students        | NULL      |
| 2           | Grade 7             | 1         |
| 3           | Grade 8             | 1         |
| 4           | Grade 7 - Section A | 2         |
| 5           | Grade 7 - Section B | 2         |
| 6           | Grade 8 - Section A | 3         |
```

```
    +-------------+---------------------+-----------+
    (6 rows)
```

KEY IDEA (what a NULL MEANS): parent_id is NULL on exactly one row, and that emptiness is not "missing" or "zero" - it MEANS "this is the top, it has no parent." The primer's Part 9 rule is to decide, per column, what each NULL stands for and be able to say it in words: here NULL = root; in E1 a blank joined_on = "join date not known"; in E6 a SET-NULL handled_by = "the staff member who handled it is gone." A NULL you cannot put into words is a design not yet finished.

The shape this describes:

```
    All Students
      |-- Grade 7
      |     |-- Grade 7 - Section A
      |     '-- Grade 7 - Section B
      '-- Grade 8
            '-- Grade 8 - Section A
```

To read each node next to its parent's NAME, join the table to ITSELF (a self-join). Use a LEFT JOIN so the root, which has no parent, still appears:

TRY IT:

```
    SELECT c.name AS child, p.name AS parent
    FROM category c
    LEFT JOIN category p ON p.category_id = c.parent_id
    ORDER BY c.category_id;

    +---------------------+--------------+
    | child               | parent       |
    +---------------------+--------------+
    | All Students        | NULL         |
    | Grade 7             | All Students |
    | Grade 8             | All Students |
    | Grade 7 - Section A | Grade 7      |
    | Grade 7 - Section B | Grade 7      |
    | Grade 8 - Section A | Grade 8      |
    +---------------------+--------------+
    (6 rows)
```

KEY IDEA: the same table appears twice in the query under two names (c for child, p for parent). The data did not change - the self-join just looks at one table from two angles.

-- The everyday query: direct children of one node -----------------------

"What are the sub-groups of All Students?" is just "which rows have parent_id = 1?"

TRY IT:

```
    SELECT name FROM category WHERE parent_id = 1 ORDER BY category_id;

    +---------+
    | name    |
    +---------+
    | Grade 7 |
    | Grade 8 |
    +---------+
    (2 rows)
```

LOOK FOR: this returns only the DIRECT children (one level down), not the

sections beneath them. Walking the WHOLE subtree needs a recursive query,
which is beyond this lesson - but the single-level question is this easy.

-- BREAK (the self-FK still enforces integrity, both ways) ----------------

First, an orphan parent. There is no category 999:

TRY IT:

```
    INSERT INTO category VALUES (7, 'Ghost Group', 999);
```

This FAILS - a parent_id must name a real category, even in the same table:

  Option A - Postgres / PGlite:

```
    +-------------------------------------------------------------------------+
    | ERROR:  insert or update on table "category" violates foreign key    |
    |         constraint "category_parent_id_fkey"                            |
    | DETAIL:  Key (parent_id)=(999) is not present in table "category".   |
    +-------------------------------------------------------------------------+
```

  Option B - SQLite (with PRAGMA foreign_keys = ON):

```
    +-------------------------------------------------------------------------+
    | Error: FOREIGN KEY constraint failed                                    |
    +-------------------------------------------------------------------------+
```

Second, deleting a parent that still has children. Grade 7 (2) has sections
4 and 5 under it:

TRY IT:

```
    DELETE FROM category WHERE category_id = 2;
```

This FAILS - the default rule will not orphan the sections, exactly like the
book/loan case in E6, except parent and child are the same table:

  Option A - Postgres / PGlite:

```
    +-------------------------------------------------------------------------+
    | ERROR:  update or delete on table "category" violates foreign key    |
    |         constraint "category_parent_id_fkey" on table "category"     |
    | DETAIL:  Key (category_id)=(2) is still referenced from table        |
    |          "category".                                                    |
    +-------------------------------------------------------------------------+
```

  Option B - SQLite (with PRAGMA foreign_keys = ON):

```
    +-------------------------------------------------------------------------+
    | Error: FOREIGN KEY constraint failed                                    |
    +-------------------------------------------------------------------------+
```

KEY IDEA: a self-reference is still a foreign key. Every rule you learned in
E6 - orphans refused, parents protected, and the ON DELETE choices - applies
to a tree too.
-- Tree delete OPTIONS: take the subtree, or lift the children ------------

For a tree you usually want one of two behaviours when a node is deleted.
Show both on scratch trees (Root -> Branch -> Leaf) so the main tree stays
intact.

(a) ON DELETE CASCADE - "delete a node and its whole subtree goes with it."

TRY IT:

```
    CREATE TABLE cat_cascade (
      id         INTEGER PRIMARY KEY,
      name       TEXT NOT NULL,
      parent_id  INTEGER REFERENCES cat_cascade(id) ON DELETE CASCADE
    );
    INSERT INTO cat_cascade VALUES (1,'Root',NULL), (2,'Branch',1), (3,'Leaf',2);

    DELETE FROM cat_cascade WHERE id = 2;     -- delete Branch
    SELECT * FROM cat_cascade ORDER BY id;
```

LOOK FOR: deleting Branch also removed Leaf (Branch's child). CASCADE follows the chain down. Only Root is left.

```
    +----+------+-----------+
    | id | name | parent_id |
    +----+------+-----------+
    | 1  | Root | NULL      |
    +----+------+-----------+
    (1 row)
```

(b) ON DELETE SET NULL - "delete a node and its children become roots."

TRY IT:

```
    CREATE TABLE cat_setnull (
      id         INTEGER PRIMARY KEY,
      name       TEXT NOT NULL,
      parent_id  INTEGER REFERENCES cat_setnull(id) ON DELETE SET NULL
    );
    INSERT INTO cat_setnull VALUES (1,'Root',NULL), (2,'Branch',1), (3,'Leaf',2);

    DELETE FROM cat_setnull WHERE id = 1;     -- delete Root
    SELECT * FROM cat_setnull ORDER BY id;
```

LOOK FOR: Root is gone; Branch's parent_id became NULL, so Branch is now a root of its own. Leaf still points at Branch, untouched.

```
    +----+--------+-----------+
    | id | name   | parent_id |
    +----+--------+-----------+
    | 2  | Branch | NULL      |
    | 3  | Leaf   | 2         |
    +----+--------+-----------+
    (2 rows)
```

Drop the scratch trees:

```
    DROP TABLE cat_cascade;   DROP TABLE cat_setnull;
-- The one thing a self-FK does NOT catch: cycles -------------------------
```

>> PITFALL: a foreign key only checks that the row you point at EXISTS. It does not check that you are pointing somewhere sensible. So the database will happily let you point the root at one of its own descendants, making a loop.

TRY IT (on the main category tree):

```
    UPDATE category SET parent_id = 4 WHERE category_id = 1;
```

There is NO error. Category 4 exists, so the FK is satisfied - but you have just built a cycle: 1 -> 4 -> 2 -> 1 -> 4 -> ... with no top and no bottom. Any code that walks "up to the root" would now loop forever. Undo it:

```
    UPDATE category SET parent_id = NULL WHERE category_id = 1;   -- 1 is root again
```

KEY IDEA: foreign keys guarantee "the parent exists," not "the tree is sane." Preventing cycles is the APPLICATION's job - check, before you set a parent, that the new parent is not the node itself or one of its descendants.

MAPS TO: this is primer Part 5.3 (self-referencing / adjacency-list trees) and primer Part 9 (ON DELETE rules, now on a self-FK). It is companion confusion 7 (how a table points at itself) and confusion 8 (why the database will not stop a cycle). In the samples PDF it is Lesson 12 (hierarchies / parent_id). The student-panel project uses exactly this adjacency-list shape for its nested GROUPS and nested PARAMETERS (each row carries a parent_id, NULL at the top), and - because of the pitfall above - it guards against cycles in its own code before saving a new parent.

-- THREE MORE PATTERNS TO COMPLETE THE SET (E9-E11) ----------

The four fixes above (E5-E8) are the ones you reach for most. Three rarer shapes from the primer's five hard patterns round out the set: a true one-to-one, a weak (dependent) entity, and a three-way (ternary) fact. Each is self-contained - it builds its own little tables and drops them at the end, so it will not disturb the running library you need for Part 5.

E9. ONE-TO-ONE: usually MERGE, split only on purpose.

>> CONTEMPLATE: a member has at most one locker; a locker belongs to one member. That is one-to-one. The primer's first advice is MERGE - just add locker columns to member - UNLESS you have a reason to split (rare, sensitive, or bulky data). When you DO split, the trick that keeps it one-to-one and not one-to-many is two words on the foreign key: UNIQUE and NOT NULL.

-- BUILD (a split 1:1, done right; reuses member2 from E5) -----------------

TRY IT:

```
    CREATE TABLE locker (
      locker_id  INTEGER PRIMARY KEY,
      member_id  INTEGER NOT NULL UNIQUE REFERENCES member2(member_id),
      shelf      TEXT NOT NULL
    );

    INSERT INTO locker VALUES
      (1, 1, 'A-12'),
      (2, 2, 'A-13');

    SELECT * FROM locker ORDER BY locker_id;
```

LOOK FOR: one locker row per member, each pointing at a member2 row.

```
    +-----------+-----------+-------+
    | locker_id | member_id | shelf |
    +-----------+-----------+-------+
    | 1         | 1         | A-12  |
    | 2         | 2         | A-13  |
    +-----------+-----------+-------+
    (2 rows)
```

-- BREAK (UNIQUE is what forbids the SECOND locker) ----------

TRY IT - try to give member 1 a second locker:

```
    INSERT INTO locker VALUES (3, 1, 'A-14');
```

LOOK FOR: refused - the UNIQUE on member_id is what makes this one-to-one and not one-to-many. Without UNIQUE this would be an ordinary 1:N (phones, E5).

```
    ERROR (Option A, Postgres / PGlite):
      ERROR:  duplicate key value violates unique constraint
      "locker_member_id_key"
      DETAIL:  Key (member_id)=(1) already exists.
    ERROR (Option B, SQLite):
      Error: UNIQUE constraint failed: locker.member_id
```

KEY IDEA: 1:1 = "merge unless you have a reason to split." If you split, put the FK on the side that must always be present and stamp it UNIQUE + NOT NULL - those two words are what turn a 1:N into a true 1:1 (NOT NULL = every locker must have a member; UNIQUE = no member gets two).

MAPS TO: primer Part 5.2 (one-to-one: usually merge, else FK on the never-empty side with UNIQUE + NOT NULL) and the mapping rule in primer Part 7 (1:1 -> FK + UNIQUE). student-panel has few true 1:1 splits; optional per-row extras tend to live in the same row instead - the primer's "merge" answer. Drop the demo:

```
    DROP TABLE locker;
```

E10. A WEAK ENTITY: an order and its lines (and a reserved-word trap).

>> CONTEMPLATE: some things cannot exist on their own. An order-LINE ("3 pencils") is meaningless without the order it sits on; "line 3" only means something as "line 3 OF order 88." That is a WEAK entity: it leans on a parent for its very existence AND for part of its identity. Note the table is named orders, not order - "order" is a reserved word (ORDER BY), so a table called order forces awkward quoting or outright breaks; the plural sidesteps it.

-- BUILD (parent + weak child; the parent is PART of the child's key) ------

TRY IT:

```
    CREATE TABLE orders (
      order_id   INTEGER PRIMARY KEY,
      placed_on  TEXT NOT NULL
    );

    CREATE TABLE order_line (
      order_id  INTEGER NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
      line_no   INTEGER NOT NULL,
      product   TEXT NOT NULL,
      qty       INTEGER NOT NULL,
      PRIMARY KEY (order_id, line_no)
    );

    INSERT INTO orders VALUES
      (88, '2026-09-30'),
      (89, '2026-10-01');

    INSERT INTO order_line VALUES
      (88, 1, 'Pencils',  3),
      (88, 2, 'Eraser',   1),
      (89, 1, 'Notebook', 2);

    SELECT * FROM order_line ORDER BY order_id, line_no;
```

LOOK FOR: line_no restarts at 1 inside each order - it is unique only WITHIN an order, which is why the key is the PAIR (order_id, line_no).

```
    +----------+---------+----------+-----+
    | order_id | line_no | product  | qty |
    +----------+---------+----------+-----+
    | 88       | 1       | Pencils  | 3   |
    | 88       | 2       | Eraser   | 1   |
    | 89       | 1       | Notebook | 2   |
    +----------+---------+----------+-----+
    (3 rows)

-- TEST (the child dies with its parent - it cannot outlive it) ------------

TRY IT - delete order 88; its two lines go with it (ON DELETE CASCADE):

    DELETE FROM orders WHERE order_id = 88;
    SELECT * FROM order_line ORDER BY order_id, line_no;

LOOK FOR: order 88's two lines are gone; only order 89's line remains. A weak
entity is the textbook case FOR cascade - a line with no order is nonsense, so
it must never be left behind.

    +----------+---------+----------+-----+
    | order_id | line_no | product  | qty |
    +----------+---------+----------+-----+
    | 89       | 1       | Notebook | 2   |
    +----------+---------+----------+-----+
    (1 row)

KEY IDEA: a weak entity gets its own table, a MANDATORY (NOT NULL) foreign key
to its parent that is also PART of its primary key, and usually ON DELETE
CASCADE - because it cannot, by definition, outlive the parent it depends on.

MAPS TO: primer Part 5.4 (weak / dependent entities) and primer Part 9 (NAMING:
avoid reserved words - order -> orders, group -> groups, user -> app_user, with
lower_snake_case throughout and thing_id for a foreign key). student-panel
difference, as ever: here order 88's lines are HARD-deleted so you can watch
CASCADE act; student-panel would SOFT-delete (set archived_at) to keep history.
Drop the demo tables (child first, then parent):

    DROP TABLE order_line;
    DROP TABLE orders;

E11. A TERNARY FACT: three parents meeting in one middle table.

>> CONTEMPLATE: "a supplier supplies a part to a project." THREE things meet in
one fact, and no single pair captures it (the same supplier may supply the same
part to a DIFFERENT project). The move is the M:N bridge, just wider: a table in
the MIDDLE holding THREE foreign keys, plus any facts of the pairing (a qty).

-- BUILD (three parents, one middle table) --------------------------------

TRY IT:

    CREATE TABLE supplier (
      supplier_id INTEGER PRIMARY KEY,
      name        TEXT NOT NULL
    );
    CREATE TABLE part (
      part_id     INTEGER PRIMARY KEY,
      name        TEXT NOT NULL
    );
    CREATE TABLE project (
      project_id  INTEGER PRIMARY KEY,
      name        TEXT NOT NULL
```

```
    );
    CREATE TABLE supply (
      supplier_id INTEGER NOT NULL REFERENCES supplier(supplier_id),
      part_id     INTEGER NOT NULL REFERENCES part(part_id),
      project_id  INTEGER NOT NULL REFERENCES project(project_id),
      qty         INTEGER NOT NULL,
      PRIMARY KEY (supplier_id, part_id, project_id)
    );

    INSERT INTO supplier VALUES (1, 'Acme'), (2, 'Globex');
    INSERT INTO part     VALUES (1, 'Bolt'), (2, 'Nut');
    INSERT INTO project  VALUES (1, 'Bridge'), (2, 'Tower');

    INSERT INTO supply VALUES
      (1, 1, 1, 500),
      (1, 1, 2, 200),
      (2, 2, 1, 500);

    SELECT * FROM supply ORDER BY supplier_id, part_id, project_id;

LOOK FOR: Acme (1) supplies the Bolt (1) to BOTH projects - two legal rows,
because the project is part of what makes each row distinct.

    +-------------+---------+------------+-----+
    | supplier_id | part_id | project_id | qty |
    +-------------+---------+------------+-----+
    | 1           | 1       | 1          | 500 |
    | 1           | 1       | 2          | 200 |
    | 2           | 2       | 1          | 500 |
    +-------------+---------+------------+-----+
    (3 rows)

-- BREAK (each of the three FKs still refuses an orphan) -------------------

TRY IT - point at a project that does not exist:

    INSERT INTO supply VALUES (1, 1, 999, 10);

LOOK FOR: refused - all three columns are foreign keys, so none may point at a
missing parent.

    ERROR (Option A, Postgres / PGlite):
      ERROR:  insert or update on table "supply" violates foreign key
      constraint "supply_project_id_fkey"
      DETAIL:  Key (project_id)=(999) is not present in table "project".
    ERROR (Option B, SQLite):
      Error: FOREIGN KEY constraint failed

KEY IDEA: a ternary (three-way) fact becomes a table in the MIDDLE with three
foreign keys - the same "middle table" move as the M:N bridge, one column wider.
If you give that middle table its own surrogate id instead, DECIDE whether the
natural TRIPLE must be UNIQUE (here: can a supplier supply the same part to the
same project twice? if not, keep it unique; if yes - e.g. on different dates -
add the date to the key, exactly as E7 widened its key for retakes).

MAPS TO: primer Part 5.5 (ternary relationships -> an entity in the middle) and
the "when a bridge/ternary gets its own id, decide the UNIQUE" rule it shares
with Part 5.1. Drop the demo tables (middle first, then the three parents):

    DROP TABLE supply;
    DROP TABLE supplier;
    DROP TABLE part;
    DROP TABLE project;
```

```
-------------------------------------------------------------------------------
 PART 5 - TEST WITH ROWS (THE DISCIPLINE BOTH BOOKS END ON)
-------------------------------------------------------------------------------

 WHAT: before you trust a design, put a handful of REAL rows in and try to
 answer the questions the design exists for. WHY: a design that cannot answer
 its own purpose questions is wrong, no matter how tidy it looks. The primer
 ends on this (Part 7 step 10, "review against the purpose"); the samples PDF
 ends on it too (Lesson 18, "test with rows"). This part is also the hands-on
 twin of the primer's Part 8 worked example (the community library drawn all the
 way down to sample rows and answered questions) - our member2/book/loan tables
 ARE that little library, and Q1-Q6 below are its questions, answered live.

 You already have good tables from E5-E8, and they already hold rows. Nothing
 new to insert - just ask. Answer each question out loud first, then run it.

   Q1 "How many loans does each member have?" (uses member2 + loan from E6)

       SELECT m.full_name, COUNT(l.loan_id) AS loans
       FROM member2 m
       LEFT JOIN loan l ON l.member_id = m.member_id
       GROUP BY m.full_name
       ORDER BY m.full_name;

     LOOK FOR: a count per member. The LEFT JOIN means a member with zero loans
     still shows (with 0) - that is the "optional side" (the little o) from the
     crow's-foot cheat-sheet, made real. Lena has no loans and still appears.

     +-------------+-------+
     | full_name   | loans |
     +-------------+-------+
     | Lena Hassan | 0     |
     | Omid Karimi | 1     |
     | Sara Noori  | 2     |
     +-------------+-------+
     (3 rows)

     >> CONTEMPLATE: swap LEFT JOIN for a plain JOIN and Lena DISAPPEARS - a
     plain (inner) join drops members with no matching loan. The word "LEFT"
     is the whole difference between "everyone, with their count" and "only
     those who borrowed." Note too that member 1 reads 'Sara Noori' here,
     because we renamed her in E6 and the loan borrows the name live.

     MAPS TO: that little "o" is the crow's-foot optional mark, now made of real
     rows. The primer's "reading & drawing ER diagrams" gives the full rule: each
     line-end carries TWO marks, read from outside in toward the box - the FAR
     mark is the MINIMUM (o = optional/zero, | = mandatory/one), the NEAR mark is
     the MAXIMUM (| = one, < = many, the crow's foot). The four right-end
     combinations are  o| zero-or-one,  || exactly-one,  o< zero-or-many,  |<
     one-or-many; the marks MIRROR (flip) on the two ends but say the same thing.
     Here MEMBER ||--o< LOAN: each loan has exactly one member, a member has zero
     or many loans - and Lena's zero is the "o" you just watched survive the
     LEFT JOIN.

   Q2 "What grade did Sara get in Algebra?" (uses enrolment from E7)

       SELECT grade FROM enrolment WHERE student_id = 1 AND course_id = 10;

     +-------+
     | grade |
     +-------+
     | A     |
```

```
  +-------+
  (1 row)
```

  LOOK FOR: one exact cell. The grade lived on the enrolment (the pairing),
  so asking for it is a plain lookup by the composite key.

Q3 "List the direct sub-groups of 'All Students'." (uses category from E8)

```
    SELECT name FROM category WHERE parent_id = 1 ORDER BY category_id;

  +---------+
  | name    |
  +---------+
  | Grade 7 |
  | Grade 8 |
  +---------+
  (2 rows)
```

  LOOK FOR: the tree answers "one level down" with a single WHERE on
  parent_id. The grandchildren (the sections) are correctly NOT listed.

Q4 "Which members have more than one phone?" (uses phone from E5)

```
    SELECT m.full_name, COUNT(p.phone_id) AS phones
    FROM member2 m
    JOIN phone p ON p.member_id = m.member_id
    GROUP BY m.full_name
    HAVING COUNT(p.phone_id) > 1;

  +------------+--------+
  | full_name  | phones |
  +------------+--------+
  | Sara Noori | 2      |
  +------------+--------+
  (1 row)
```

  LOOK FOR: HAVING filters AFTER grouping - it keeps only the groups whose
  count passes the test. (WHERE filters rows BEFORE grouping; HAVING filters
  the groups themselves.) Only Sara has two phones, so only Sara qualifies.

Q5 "How many times has each book been borrowed - including the ones that
    never were?" (uses book + loan from E6)

```
    SELECT b.title, COUNT(l.loan_id) AS times_borrowed
    FROM book b
    LEFT JOIN loan l ON l.book_id = b.book_id
    GROUP BY b.title
    ORDER BY b.title;

  +---------------+----------------+
  | title         | times_borrowed |
  +---------------+----------------+
  | Open Sky      | 0              |
  | Quiet Rivers  | 1              |
  | The Blue Door | 2              |
  +---------------+----------------+
  (3 rows)
```

  LOOK FOR: 'Open Sky' shows 0, not a missing row. This is why we re-added
  it in E6 - a never-borrowed book is a real answer, and only the LEFT JOIN
  keeps it. Count COUNT(l.loan_id), not COUNT(*): COUNT(*) would wrongly
  count the one all-NULL row the LEFT JOIN makes for Open Sky as 1.

Q6 "Who is on the Algebra roster, and with what grade?" (uses enrolment + student)

```
      SELECT s.full_name, e.grade
      FROM enrolment e
      JOIN student s ON s.student_id = e.student_id
      WHERE e.course_id = 10
      ORDER BY s.full_name;

    +-------------+-------+
    | full_name   | grade |
    +-------------+-------+
    | Omid Karimi | B     |
    | Sara Ahmadi | A     |
    +-------------+-------+
    (2 rows)

    LOOK FOR: here Sara is 'Sara Ahmadi', NOT 'Sara Noori'. The courses live
    in a DIFFERENT table (student) from the library members (member2); we only
    renamed the library row. Same person in your head, two separate rows in
    two separate tables - a good reminder that a name is not an identity.
-- Now feel the contrast: ask a BAD table the same kind of question --------

>> CONTEMPLATE: each question above was easy because the design put each fact
in one clear place. Go back to the BAD table from E3 (student_bad_subjects,
the comma-list) and try "who takes Science?" - feel how it has no clean
answer. That contrast IS the lesson.

TRY IT (the exact-match attempt):

    SELECT name FROM student_bad_subjects WHERE subject = 'Science';

    +------+
    | name |
    +------+
    +------+
    (0 rows)

Zero rows - nobody's cell is EXACTLY the text 'Science'; they are
'Social Science', 'Math, Science', 'History, Science'. So you reach for LIKE:

TRY IT (the substring attempt):

    SELECT name FROM student_bad_subjects WHERE subject LIKE '%Science%';

    +------+
    | name |
    +------+
    | Sara |
    | Omid |
    | Lena |
    +------+
    (3 rows)

LOOK FOR: now it is WRONG the other way. Sara's cell is 'Social Science', not
Science at all, but '%Science%' matched it. The comma-list cannot tell "takes
Science" from "takes Social Science." The bridge table in E7 answered the
same question exactly, with no guessing. THAT is the payoff of the design.

KEY IDEA: "test with rows" is not an afterthought - it is how you know a
design is finished. If a few real rows can answer your purpose questions
cleanly - and the BAD design visibly cannot - the good design is sound.
-------------------------------------------------------------------------------
PART 6 - MAP BACK: EXERCISES vs PRIMER vs PDF vs STUDENT-PANEL
-------------------------------------------------------------------------------
```

One table so every exercise has a home in your other materials, and every
place the PRACTICE deliberately differs from student-panel is named.

Exercise	Idea	Primer	Samples PDF	student-panel
E1 warm-up	table/row/column	Part 2	Lesson 1-2	same idea
E2 phone1/2/3	repeating columns	Part 6	Lesson 14	avoided
E3 list-in-cell	1NF / atomic value	Part 6	Lesson 14	avoided
E4 wide table	update anomaly	Part 6	Lesson 16	avoided
E5 phone table	multi-valued -> table	Part 2.2/5	Lesson 3	same shape
E6 split+FK	split + FK + ON DELETE	Part 5/9	Lesson 6-8/16	same shape
E7 enrolment	M:N bridge + key	Part 5.1/3	Lesson 9-11	= evaluations
E8 category tree	self-reference tree	Part 5.3/9	Lesson 12	groups+params
E9 one-to-one	1:1 split + UNIQUE	Part 5.2	-	mostly merged
E10 order-line	weak entity + CASCADE	Part 5.4/9	-	weak children
E11 supply	ternary -> middle	Part 5.5	-	score triple

Read it as: "the thing I practised in Ex N is explained in primer Part P and
PDF Lesson L, and shows up in student-panel as the last column."
-- Cheat-sheet 1: the five constraints and what each one PROMISES ----------

Each is a rule the database keeps for you. The right-hand note is the error
you saw (or would see) when it is broken.

  PRIMARY KEY .... "every row has one id, unique and never NULL."
                      break it with a duplicate id -> "duplicate key value
                      violates unique constraint" (PG) / "UNIQUE constraint
                      failed" (SQLite). Seen in E1 and E7.

  NOT NULL .... "this column must always have a value."
                      break it with a missing value -> "null value in column
                      ... violates not-null constraint" (PG) / "NOT NULL
                      constraint failed" (SQLite). Seen in E1.

  UNIQUE .... "no two rows may share this value."
                      break it with a repeat -> same wording as PRIMARY KEY's
                      duplicate error. Seen in E1 (card_no).

  FOREIGN KEY .... "this value must name a row that really exists in the
                      parent table." break it with an orphan -> "violates
                      foreign key constraint ... is not present" (PG) /
                      "FOREIGN KEY constraint failed" (SQLite). Seen in
                      E5, E6, E7, E8.

  COMPOSITE PK ...... "the COMBINATION of these columns is unique (each column
                      alone may repeat)." break it with a repeated pair ->
                      the duplicate-key error naming all key columns. Seen
                      in E7 (student_id + course_id).

-- Cheat-sheet 2: the ON DELETE rules, as plain sentences -----------------

When you delete a PARENT row, the FK on the child decides what happens:

  (default / NO ACTION / RESTRICT)
      "You may NOT delete a parent while any child still points at it."
      Delete is refused. You saw this in E6 (book 100) and E8 (Grade 7).

  ON DELETE RESTRICT
      The same block, written out loud. Saw it in E6 (member_r).

```
  ON DELETE CASCADE
      "Delete the parent and its children are deleted too" - following the
      chain all the way down. Saw it in E6 (member_c -> phones) and E8
      (cat_cascade -> the whole subtree).

  ON DELETE SET NULL
      "Delete the parent and the children SURVIVE, but their pointer is
      emptied to NULL." The FK column must be nullable (no NOT NULL). Saw it
      in E6 (staff_s -> loan handled_by) and E8 (cat_setnull -> child becomes
      a root).

>> PITFALL (worth repeating): in SQLite none of the FK rules fire unless you
ran PRAGMA foreign_keys = ON; first. Postgres and PGlite always enforce them.

-- Cheat-sheet 3: the three normal forms, each as one promise -------------

Recite this over every column, the whole of normalisation in a sentence:
each non-key fact must depend on "the key, the whole key, and nothing but
the key." (primer PART 6 - NORMALISATION.)

  1NF = "the key" ......... one value per cell: no comma-lists, no
                            phone1/phone2 repeat-columns. A repeating thing
                            gets its own child table. Fixed in E2 and E3.

  2NF = "the whole key" ... on a COMPOSITE key, no column may depend on only
                            PART of it (only bites when the key has more than
                            one column). Seen in E7 (student_name hung off
                            student_id alone, half the key, and drifted).

  3NF = "nothing but" ..... no non-key column may depend on another non-key
                            column; lift the derived group into its own
                            table. Fixed in E4 and E6.

>> DENORMALISATION is breaking 3NF on PURPOSE for read speed (say, storing a
   total you could re-add). A knowing tradeoff, not a mistake - but only with
   eyes open. (primer PART 6, and "derive, don't store" in Part 2 / Part 9.)

>> A FROZEN FACT is NOT denormalisation: a score or grade as it stood on a
   date (E5, E7) is a real time-stamped observation you must keep, not a
   cached copy of a live value. Keep it; do not "normalise it away."

-- Cheat-sheet 4: a self-check rubric - tick each with a reason -----------

Run any schema you draw past these before you trust it (this mirrors the
primer's PART 10 self-check rubric; the right-hand note is the exercise that
drilled each one here):

  [ ] every table has one primary key that never changes ...... E1
  [ ] every cell holds ONE value - no comma-lists, no phoneN .. E2, E3
  [ ] no fact is stored twice where the copies can drift ...... E4
  [ ] every foreign key points at a row that really exists .... E5, E6
  [ ] each FK has a CHOSEN ON DELETE rule, not an accident .... E6
  [ ] every M:N lives in its own bridge with a key ............ E7
  [ ] a hierarchy uses a self-reference, root = NULL .......... E8
  [ ] a 1:1 is merged, or split with FK UNIQUE + NOT NULL ..... E9
  [ ] a weak/child part sits in its own table, parent in key .. E10
  [ ] a three-way link is a middle table with three FKs ....... E11
  [ ] names dodge reserved words; lower_snake_case, thing_id .. Part 6
  [ ] every decision you were UNSURE about is written down .... bring it!

WHERE PRACTICE DIFFERS FROM STUDENT-PANEL (our standing rule - always flagged):

  - IDS: practice uses hand-typed INTEGER ids (1, 2, 3...) so you can read and
```

```
    type them. student-panel uses UUID primary keys (long random values) plus
    a unique human-friendly code (like a student_code). Same purpose: one
    stable key per row that never changes, even if a name does.

  - DELETE: practice HARD-deletes rows, so you can watch referential integrity
    and the ON DELETE rules bite. student-panel SOFT-deletes - it stamps an
    archived_at timestamp and hides the row instead of removing it, so history
    and past scores are never lost. Same goal (no dangling references),
    gentler mechanism.

  - ENGINE / DIALECT: practice may use SQLite (Option B); student-panel uses
    Postgres (Option A matches it, via PGlite in the browser). Watch the
    RESERVED WORDS - order, group and user are reserved, so name the tables
    orders, groups and app_user (the fix primer PART 9 - DESIGN JUDGEMENT
    teaches). Keep every name lower_snake_case and spell a foreign key
    thing_id (student_id, book_id), as both the primer and student-panel do.
    You met this in E10 (order -> orders). And remember SQLite's
    foreign_keys-ON pragma from E6.

  - MODEL DETAIL: practice collapsed book + physical-copy into one `book`
    table, and used a single `enrolment` grade. student-panel's real design is
    fuller (e.g. parameters and groups are each their own nested tree, and a
    score is a frozen fact on the student x parameter x period pairing). The
    MOVES are the same; student-panel just has more of them.

  - SCALE: practice tables are tiny teaching toys (three members, three
    books). student-panel is a full multi-table design. Identical shapes,
    different counts.

-- Reset the sandbox (optional): drop everything, children before parents ---

If you want a clean slate, run these in order (a referencing table must go
before the table it points at, the same RESTRICT rule you saw for rows):

    DROP TABLE loan;                -- points at member2 and book
    DROP TABLE phone;               -- points at member2
    DROP TABLE enrolment;           -- points at student and course
    DROP TABLE book;
    DROP TABLE member2;
    DROP TABLE student;
    DROP TABLE course;
    DROP TABLE category;            -- self-reference; no other table needs it
    DROP TABLE loan_wide;           -- the bad tables from Part 3/E4
    DROP TABLE member_bad_list;
    DROP TABLE member_bad_phones;
    DROP TABLE student_bad_subjects;
    DROP TABLE member;              -- the E1 warm-up table

(The scratch tables - member_r, loan_r, member_c, phone_c, staff_s, loan_s,
enrolment2, cat_cascade, cat_setnull - were already dropped inside E6-E8.)

KEY IDEA: you have now physically built, in miniature, four of the exact
shapes student-panel is made of - a child table with a foreign key (E5), a
3NF split with referential integrity (E6), a many-to-many bridge (E7), and a
self-reference tree (E8). Re-deriving student-panel later is mostly these
same moves, repeated with care.
-------------------------------------------------------------------------------
WHERE THIS LEADS
-------------------------------------------------------------------------------

Keep the sandbox tab and your paper sheets. Our next step (after you have read
the primer, the companion, and done this practice, and brought me your
questions) is to re-derive the student-panel schema together using the
```

primer's ten-step method - and you will recognise E6, E7, and E8 inside it.

What you can now do, hands-on, from nothing:
- make a table with the right constraints (E1);
- recognise the three classic shape mistakes and WHY they hurt (E2-E4);
- give a repeating thing its own child table (E5);
- split a wide table and connect it with a foreign key, choosing the right ON DELETE rule (E6);
- model a many-to-many with a bridge table and a composite key (E7);
- model a hierarchy with a self-reference, and know the cycle gap (E8);
- handle the three rarer shapes - a 1:1, a weak child table, and a ternary link (E9-E11);
- and TEST any design by asking it real questions (Part 5).

Before we meet, the primer's PART 10 - PRACTICE asks you to run the full ten-step method on ONE design from scratch - a gym, a recipe app, or a ticketing system - and to CRITIQUE a deliberately bad one-table school design, naming its anomalies and redrawing it. You have already done that critique by hand here: E2, E3 and E4 ARE that broken design taken apart a piece at a time; bring those tables (and the self-check rubric above) to the from-scratch attempt.

Reminder of the rule we never break: we do NOT change the student-panel project. You build your own version in student-panel-I, and wherever your way differs from student-panel's way, I will say so - just as this lesson flagged ids, deletes, dialect, and model detail above.

Take your time. Bring questions. A confused question now saves a broken table later.

- end of practice lesson -